INF-253 Lenguajes de Programación
Tarea 3 2026-2: El regreso de los Javalings

Profesores:Wladimir Ormazábal Orellana
Ayudante coordinador: Bastián Ríos Lastarria

24 de septiembre de 2026

Índice

1. Contexto **2**

2. Descripción General **2**

3. Arquitectura y Modelado (Requisitos Obligatorios) **2**
3.1. Paquetes (`packages`) . . . 2
3.2. Herencia y Clases Abstractas . . . 2
3.3. Interfaz y Clases Anidadas . . . 2
3.4. Encapsulamiento . . . 3
3.5. Diagrama de clases . . . 3

4. Mecánicas del Simulador **3**
4.1. Tabla de Tipos Elementales . . . 3
4.2. Fórmula de Daño . . . 4
4.3. Características de un Javaling . . . 4
4.4. Heurística para decidir . . . 4

5. Formato de Entrada y Salida **5**
5.1. Formato de Entrada (Archivos JSON) . . . 5
5.1.1. 1. Catálogo de Movimientos (`movimientos.json`) . . . 5
5.1.2. 2. Bracket de Equipos (`equipos.json`) . . . 6
5.2. Salida (Output) . . . 6

6. Sobre la entrega **8**

7. Calificación **8**
7.1. Entrega . . . 8
7.1.1. Resto del puntaje (Total 70 pts) . . . 9
7.2. Descuentos . . . 9

1. Contexto

Tras tu anterior éxito con *PhotoChop*, unos emprendedores sansanos han visto tu proyecto y maravillados, han decidido contratarte en el equipo de balance para ayudarlos a crear la secuela a un anterior proyecto sansano conocido como Javaling. Sin embargo, ahora mismo la secuela se encuentra muy desbalanceada en términos de poder de los Javaling, por lo que te solicitan a ti crear un simulador de combate estilo torneo que permita determinar la victoria de cada equipo y así visualizar qué Javalings están muy fuertes (o débiles).

2. Descripción General

Deberás construir un 'Juez Automático' para un torneo de criaturas elementales llamadas "javalings". El sistema recibirá un archivo de entrada con los equipos participantes, organizará un torneo de eliminación directa (con llaves en potencias de 2: 2, 4, 8, 16 o 32 equipos) y simulará los combates de manera 100 % determinista mediante una heurística predefinida, escribiendo el resultado jugada a jugada en un archivo de salida.

3. Arquitectura y Modelado (Requisitos Obligatorios)

El proyecto debe dividirse estrictamente en los siguientes paquetes y estructuras:

3.1. Paquetes (`packages`)

- `Entidades`: Contendrá las entidades puras (Javaling, Equipo).
- `MecanicasCombate`: Contendrá los ataques, tipos y sus interacciones, etc.
- `Simulador`: Aquí se encuentra la lógica y gestión del torneo, así como los registros y escritura.

3.2. Herencia y Clases Abstractas

Debe existir una clase abstracta `Ataque` que defina el comportamiento base de los movimientos. De ella deben heredar tres subclases:

- `AtaqueFisico`: Utiliza la estadística de Ataque del agresor y Defensa del defensor.
- `AtaqueEspecial`: Utiliza el Ataque Especial del agresor y Defensa Especial del defensor.
- `AtaqueEstado`: No causa daño directo. Aplica un multiplicador (ej. 1,5 o 2,0) a una estadística propia.

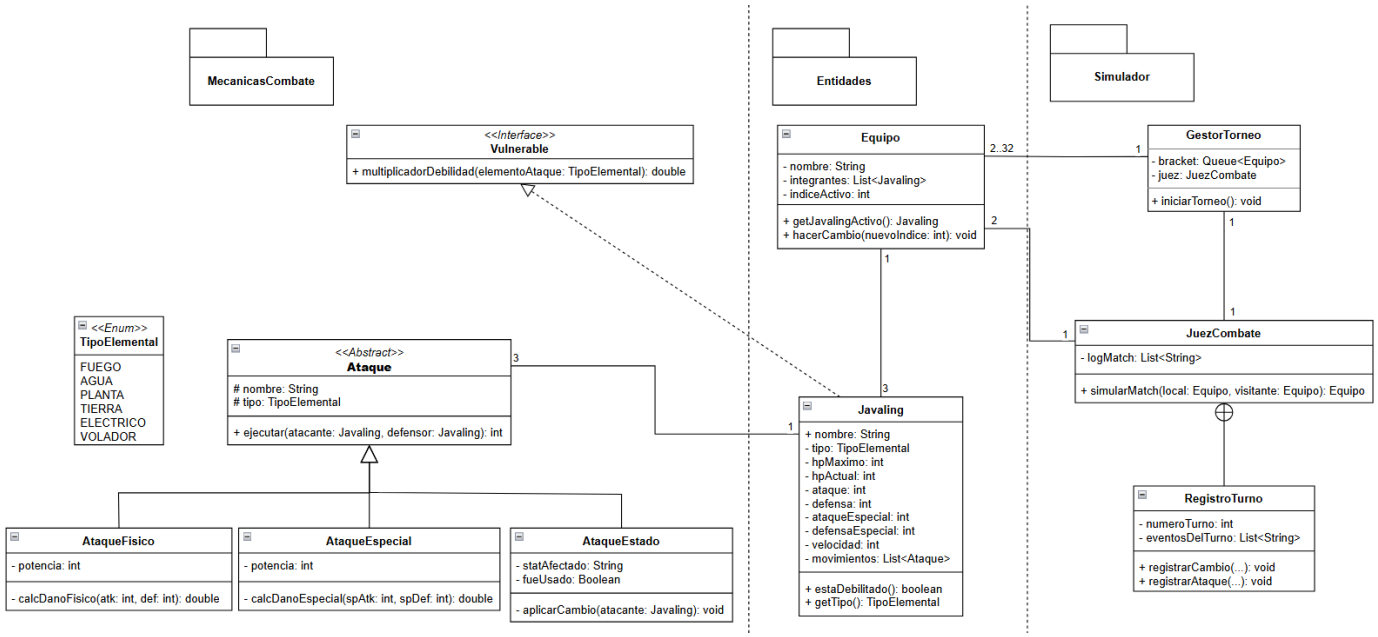
3.3. Interfaz y Clases Anidadas

- **Interfaz:** Las criaturas deben implementar la interfaz `Vulnerable`, la cual obliga a sobreescribir el método `multiplicadorDebilidad` el cual debe contener la lógica de la **Tabla de tipos (sección 4.1)**.
- **Clase Anidada:** Dentro de la clase `JuezCombate`, debe existir una clase anidada privada llamada `RegistroTurno`, encargada de registrar y anotar las acciones realizadas en cada turno, logrando separacion de responsabilidades y dejando al JuezCombate manejar la lógica sin tener que escribir los registros también.

3.4. Encapsulamiento

Notar que distintos atributos poseen distintos símbolos, se debe respetar estrictamente la simbología utilizada en el diagrama. El único atributo público es el nombre de los Javaling, el resto debe permanecer en *Private*, a excepción de los *Protected* en la clase *Abstracta* **Ataque**.

3.5. Diagrama de clases



4. Mecánicas del Simulador

4.1. Tabla de Tipos Elementales

El sistema cuenta con 6 tipos. Los multiplicadores de daño se aplican según la siguiente matriz:

Atacante	Efectivo contra (x2.0)	Débil contra (x0.5)	Inmune a (x0.0)
Fuego	Planta	Agua, Fuego, Tierra	-
Agua	Fuego, Tierra	Agua, Planta	-
Planta	Agua, Tierra	Fuego, Planta, Volador	-
Tierra	Eléctrico, Fuego	Planta	Volador
Eléctrico	Agua, Volador	Planta, Eléctrico	Tierra
Volador	Planta	Eléctrico	-

4.2. Fórmula de Daño

El daño infligido por ataques físicos y especiales se debe calcular todo como decimal y aplicar un truncado solo al resultado final.

$$\text{Daño} = \left\lfloor \frac{\text{Potencia} \times \left(\frac{\text{Estadística Ofensiva del atacante}}{\text{Estadística Defensiva del receptor}} \right) \times \text{Modificador de Tipo} \times \text{STAB}}{2} \right\rfloor$$

STAB: *Same Type Attack Bonus* - x1.5 de Bonus si el ataque realizado corresponde al mismo tipo que el Javaling que está atacando (Es decir, un Javaling tipo *Fuego* pegará x1.5 veces más con un ataque de tipo Fuego que con un ataque de tipo Agua si ambos ataques tienen la misma potencia).

4.3. Características de un Javaling

Cada combatiente del torneo, denominado **Javaling**, posee un conjunto de características que definen su desempeño. Todo Javaling debe instanciarse obligatoriamente con los siguientes atributos:

- **Tipo Elemental:** Un único tipo (Fuego, Agua, Planta, Tierra, Eléctrico o Volador) que define sus debilidades, resistencias y el bono de daño por compatibilidad (STAB).
- **Estadísticas (Stats):**
  - **HP (Puntos de Vida):** Define la cantidad de daño máximo que la unidad puede recibir antes de ser derrotada. Se debe distinguir entre el HP Máximo y el HP Actual.
  - **Ataque Físico y Defensa Física:** Valores utilizados exclusivamente en el cálculo de daño cuando se ejecuta un `AtaqueFisico`.
  - **Ataque Especial y Defensa Especial:** Valores utilizados exclusivamente en el cálculo de daño cuando se ejecuta un `AtaqueEspecial`.
  - **Velocidad:** Determina el orden de acción en cada turno. El Javaling con mayor velocidad ejecuta su movimiento primero. En caso de empate, el equipo "local"(seleccionado como primero) atacará antes.
- **Set de Movimientos:** Una estructura de datos (arreglo o lista) que debe contener exactamente **3 ataques**, los cuales pueden ser una combinación libre de ataques físicos, especiales o de estado.

4.4. Heurística para decidir

Los Javaling actúan automáticamente basándose en la siguiente jerarquía estricta. Todo combate es evaluado turno a turno:

1. **Evaluación Activa:** El simulador calcula el daño proyectado de los 3 ataques del Javaling activo en cancha. Se ejecuta aquel ataque que reduzca más del 50 % del HP actual del rival. Si más de un ataque cumple esta condición, el que haga mayor daño total.
2. **Evaluación de Banca (Cambio de Javaling):** Si ningún ataque cumple la condición anterior, se itera sobre los compañeros de equipo vivos. Se busca un compañero cuyo mejor ataque supere el 50 % de la vida actual del rival.

3. **Resolución de Banca:**
    - Si existe al menos un compañero válido, se realiza el cambio (se pierde el turno de ataque) y el daño del rival lo recibe el Javaling entrante.
    - Si hay múltiples candidatos viables, se prioriza el que cause mayor daño numérico.
    - En caso de empate de daño proyectado en banca, se elige al primer candidato según su índice en la formación del equipo.
4. **Último Recurso:** Si ningún aliado en banca cumple la condición anterior, el Javaling en cancha utilizará su `AtaqueEstado` si posee uno y no lo ha usado (**solo un uso**). En caso contrario, lanzará su ataque que genere mayor daño numérico. Como última opción si ninguna de las anteriores condiciones se llegase a cumplir, se realiza el primero de los 3 ataques con los que cuenta el Javaling.
5. En caso de que el ataque de un Javaling reduzca la vida del contrincante a 0, ingresa automáticamente el primer Javaling vivo de la lista del equipo a reemplazarlo.

Nota de Desempate de Turno por Regla de Impar: Si ambos Javaling en cancha poseen la misma estadística de Velocidad al inicio del turno, o la heurística decide que ambos equipos cambian su Javaling en campo, el equipo 'impar' siempre actúa primero, es decir, entre equipo 1 vs equipo 2, siempre equipo 1 actúa antes que equipo 2 en caso de empate de decisiones.

5. Formato de Entrada y Salida

5.1. Formato de Entrada (Archivos JSON)

El programa recibirá por argumento la ruta a **dos archivos JSON**: un catálogo de movimientos y la lista de equipos.
El sistema deberá parsear primero el catálogo y almacenar los objetos instanciados (se sugiere el uso de un `Map` o `HashMap`), para luego asignar las referencias correspondientes al leer los equipos.

5.1.1. 1. Catálogo de Movimientos (`movimientos.json`)

Este archivo contiene la definición de todos los ataques que existen en la simulación. El campo `categoria` determina qué subclase de `Ataque` debe instanciarse (`Fisico`, `Especial` o `Estado`).

```
[
  {
    "nombre": "Pájaro Osado",
    "categoria": "Físico",
    "elemento": "Volador",
    "potencia": 120
  },
{
// ...
},
  {
    "nombre": "Danza Espada",
    "categoria": "Estado",
    "stat_afectado": "ataque",
    "multiplicador": 2.0
```

```
  }
]
```

**Considerar que los movimientos tanto físicos como especiales tendrán la misma estructura en el JSON, solo la estructura del movimiento de Estado es distinta puesto que genera cambios*

5.1.2. 2. Bracket de Equipos (`equipos.json`)

Este archivo define los equipos participantes (en potencias de 2). Cada equipo posee **3 Javalings**, y cada uno referencia exactamente **3 movimientos** a través de su nombre en formato `String`. El estudiante debe ser capaz de buscar dicho nombre en su catálogo previamente cargado e inyectar el objeto `Ataque` correcto en el Javaling.

```
[
  {
    "nombre_equipo": "Team Magma",
    "javalings": [
      {
        "nombre": "Talonflame",
        "tipo": "Fuego",
        "stats": {
          "hp": 78,
          "ataque": 81,
          "defensa": 71,
          "ataque_especial": 74,
          "defensa_especial": 69,
          "velocidad": 126
        },
        "movimientos": [
          "Pájaro Osado",
          "Envite Igneo",
          "Danza Espada"
        ]
      },
      // ... (los otros 2 Javalings de Team Magma)
    ]
  },
  {
    "nombre_equipo": "Team Aqua",
    "javalings": [
      // ... (Estructura idéntica para los 3 Javalings rivales)
    ]
  }
]
```

5.2. Salida (Output)

El simulador debe generar un archivo `resultados_torneo.txt` detallando el progreso del bracket, la resolución jugada a jugada de cada enfrentamiento y la coronación del equipo ganador absoluto. Las etiquetas obligatorias y su formato esperado son:

- `[MATCH_INICIO]` Indica el inicio de un enfrentamiento del torneo.

- `Turno X:` Indica el turno en juego. El turno siempre contiene una única acción ([ATAQUE], [ESTADO] o [CAMBIO]) de cada equipo y acciones adicionales como [KO] o [ENTRA] según lo que suceda en el combate.
- `[ENTRA]` Indica el Javaling que está en combate en el momento. El turno 1 siempre contiene 2 [ENTRA] señalando el Javaling inicial. El Javaling inicial corresponde al primero en la lista de 3 Javalings de cada equipo.
- `[CAMBIO]` Un equipo retira a su Javaling activo para enviar a otro según la heurística. Siempre va al inicio del turno y sucede antes que cualquier ataque rival.
- `[ATAQUE]` Detalla el uso de un `AtaqueFisico` o `AtaqueEspecial`, indicando el atacante, el defensor, el movimiento, el daño infligido y los Puntos de Vida restantes del defensor.
- `[ESTADO]` Detalla el uso de un `AtaqueEstado`, indicando la estadística afectada y el multiplicador aplicado.
- `[KO]` Registra explícitamente qué Javaling asestó el golpe final a un Javaling rival. Ocurre cuando la vida restante de un Javaling llega a 0.
- `[MATCH_FIN]` Declara el final de un combate, el equipo ganador y el marcador final reflejado en los Javalings vivos de cada equipo (ej. $2 - 0$ corresponde a que el Equipo 1 gana con 2 Javalings restantes).
- `[TORNEO_FIN]` Anuncia al equipo campeón absoluto tras finalizar todas las llaves.

A continuación, se presenta un extracto exacto de cómo debe estructurarse la salida en consola para un enfrentamiento:

```
[MATCH_INICIO] Enfrentamiento 1: Team A vs Team B
Turno 1:
[ENTRA] Team A envía a JavalingA.
[ENTRA] Team B envía a JavalingB.
[ATAQUE] JavalingA ataca a JavalingB con Movimiento1 | Daño: 60 | vida
   restante de JavalingB: 5
[ESTADO] JavalingB usa Movimiento3 (estado) | defensa de JavalingB
   modificada por x1.5. Defensa actual: 105.
Turno 2:
[CAMBIO] Team B retira a JavalingB y envia a JavalingC
[ATAQUE] JavalingA ataca a JavalingC con Movimiento1 | Daño: 30 | vida
   restante de JavalingC: 100
[ATAQUE] JavalingC ataca a JavalingA con Movimiento2 | Daño: 78 | vida
   restante de JavalingA: 0
[KO] JavalingC ha derrotado a JavalingA.
[ENTRA] Team A envía a JavalingD.
Turno 3:
...
[MATCH_FIN] Ganador: Team B | Marcador final: 0 - 2
[MATCH_INICIO] Enfrentamiento 2: Team C vs Team D
...
[TORNEO_FIN] Campeón del torneo: Team B
```

Considerar que el ejemplo no necesariamente refleja lo que debe ocurrir en un enfrentamiento normal sino meramente ilustrativo de los distintos tipos de etiquetas y el formato solicitado, por

ejemplo, en el turno 1 Team B utiliza un movimiento de estado teniendo a JavalingC que derrota a JavalingA en un solo ataque.
Nota de evaluación: El respeto estricto de este formato de salida es un criterio de calificación obligatorio. El no cumplimiento de este formato y/o la información requerida podría conllevar a descuentos en la nota. Lo mismo aplica al uso de los métodos y atributos de la forma en la que están en el diagrama.

6. Sobre la entrega

- El código debe venir indentado y ordenado.
- Se deben entregar los siguientes archivos:
  - `README.txt` o `README.md`
  - `Makefile`
  - Todos los archivos `.java` del diagrama distribuidos en sus respectivas carpetas de paquetes.
- Los métodos, a excepción de los *getters* y *setters*, deben ir comentados, explicando clara y brevemente lo que realizan, los parámetros que reciben y los que devuelven.
- **El uso de IA está prohibido.** Cualquier sospecha de uso de herramientas de inteligencia artificial podrá conllevar descuentos en la evaluación o citación a defensa. En caso de comprobarse, la entrega será evaluada con nota 0 y notificada al profesor.
- La entrega debe realizarse en un archivo comprimido en `.tar.gz` y debe llevar el nombre: `Tarea3LP_RolAlumno.tar.gz`. Ej.: `Tarea3LP_202573000-k.tar.gz`.
- El archivo `README` debe contener el nombre y rol del alumno e instrucciones detalladas para la correcta compilación y ejecución de su programa.
- La entrega será vía Aula y el plazo máximo de entrega es hasta el día **Lunes 19 de Octubre, 23:59 horas**.
- Por cada día de atraso se descontarán 20 puntos (10 puntos si es dentro de la primera hora).
- Solo se pueden realizar consultas respecto a la tarea hasta 2 días antes de la entrega. Las consultas sobre el enunciado deben realizarse mediante el foro de la tarea disponible en Aula.

7. Calificación

7.1. Entrega

Para la calificación de su tarea, debe cumplir con un piso de requerimientos mínimos que le otorgarán **30 puntos base**. El puntaje restante (hasta 70 puntos) se evaluará según el cumplimiento técnico de la arquitectura y la lógica requerida.

- **(30 pts)** El sistema lee correctamente los archivos `JSON`, instanciando con éxito los objetos `Equipo` y `Javaling`, y se utiliza la clase anidada `RegistroTurno` para capturar e imprimir con éxito el flujo de un combate entre 2 equipos, utilizando lo extraído anteriormente.

Nota: Este puntaje evalúa que el programa compile, ejecute y logre procesar la información de entrada para generar una salida estructurada, independientemente de si los cálculos matemáticos de la heurística están 100 % correctos.

7.1.1. Resto del puntaje (Total 70 pts)

- **Heurística de Decisión y Cambios (15 pts):** El simulador respeta estrictamente el orden de decisiones: evalúa primero si un ataque activo quita más del 50 % del HP actual, luego evalúa la banca para forzar un cambio si hay un movimiento mejor, y finalmente recurre al ataque de estado o al ataque más dañino como último recurso.
- **Ataques Físicos y Especiales (15 pts):** La clase abstracta `Ataque` y sus subclases de daño calculan el daño correctamente, diferenciando el uso de estadísticas físicas (Ataque/Defensa) y especiales (Ataque Especial/Defensa Especial), además de calcular e incluir el multiplicador STAB.
- **Ataques de Estado (10 pts):** Se implementa el `AtaqueEstado` alterando correctamente la estadística indicada del Javaling. Se respeta y programa la restricción funcional de que este tipo de ataque tiene un solo uso por Javaling en todo el combate.
- **Interfaz y Tabla de Tipos (10 pts):** La clase `Javaling` implementa la interfaz `Vulnerable` devolviendo el multiplicador exacto (2,0, 0,5 o 0,0) según la tabla de tipos elementales solicitada al recibir un ataque.
- **Manejo de KOs y Reemplazos (10 pts):** El simulador detecta correctamente cuando el HP de un Javaling llega a 0, registrando el `[KO]` e ingresando automáticamente a la batalla al primer Javaling vivo disponible en la lista del equipo.
- **Prioridad de Turno y Empates (5 pts):** El Juez ordena las acciones de cada turno utilizando la estadística de Velocidad de los Javalings y resuelve de manera correcta los casos de empate según lo solicitado.
- **Registro de Turno Detallado (5 pts):** La clase anidada `RegistroTurno` posee métodos distintos y específicos para cada evento (`registrarAtaque`, `registrarCambio`, `registrarEstado`, `registrarKO`), aislando la concatenación de *Strings* de la lógica matemática del combate.

7.2. Descuentos

- Falta de `Makefile` o no funcional (-100 pts)
- Código no compila (-100 pts)
- Uso de librerías no estándar de Java (-100 pts)
- Falta de clases y/o paquetes indicados en el diagrama UML (-100 pts)
- Falta de atributos y/o métodos mínimos obligatorios del diagrama (-20 pts c/u)
- Falta de clases dentro de su package correspondiente (-20 pts c/u)
- Interfaz de consola confusa, inestable o que no respeta las etiquetas de salida exigidas (máx -30 pts)

- Atributo público que no corresponde a la rúbrica (máx -30 pts, -5 pts c/u)
- Falta de archivo `README` (-20 pts)
- Falta de información obligatoria en el `README` (nombre, rol, instrucciones) (-5 pts c/u)
- Falta de orden o mala indentación en el código fuente (-5 a -20 pts según gravedad)
- Falta de comentarios explicativos sobre cada método implementado (-5 pts c/u, máx -30 pts)
- Advertencias (*Warnings*) de compilación (-5 pts c/u)
- Incumplimiento de instrucciones (-5 a -50 según gravedad)
- Mal formato en el nombre de los archivos entregados (-5 pts c/u; -10 pts si es el archivo `.tar.gz`)
- Uso de código y/o texto generado por IA implicará un 0 y se le informará a las respectivas autoridades.
- Entrega tardía (-10 pts si es dentro de la primera hora; -20 pts por cada día o fracción de retraso). En caso de existir nota negativa, esta será reemplazada por un 0.